

ML Specification Report — playground-series-s3e9

Concrete Compressive Strength Prediction · our from-scratch agent (GBDT pool + tree-search v3)

Figures grounded in the competition's `facts.json`, `eda_summary.json`, `STATUS.md`, and `experiments_tree_v3.json`, with external benchmark facts from `/tmp/ext_facts.json`.

Overview

The task is to predict the compressive strength of concrete mixes from their composition and curing time (metric: RMSE, minimize). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (LightGBM / XGBoost / CatBoost) refined by a multi-round linear self-improvement loop and a 26-node tree search that converges on a blend. Our committed champion scores **CV OOF RMSE 12.07003**; the NVIDIA reproduce-agent, copying the public `ambrosm/pss3e9-winning-model` kernel verbatim, reaches **12.026** — a marginal edge that the four-axis evaluation scores as a **tie**.

Why it matters. Compressive strength is the key safety property of structural concrete; predicting it from mix composition cuts costly physical testing and supports engineering design.

Data

Purpose of Data. Predict `Strength`, the compressive strength of a concrete specimen — a continuous-regression Playground Series episode built on the classic concrete-mixture dataset. **Data Format** is clean **tabular CSV**, entirely numeric (no categorical columns). **Data Volume** is **5,407 training rows / 3,605 test rows**, with 8 raw predictive features once `id` and the target are removed.

Data Quality is high on the surface — **no missing values** in either split and no strong covariate shift (largest train ↔ test mean shift: `AgeInDays` -5.02%, `BlastFurnaceSlag` -4.79%) — but the defining wrinkle is **label noise: 2,401 train rows (44.4%) are non-first exact duplicates** of another row's 8-feature tuple, each carrying a *different* measured `Strength`, so ~56% of rows belong to some duplicate group. This is measurement/synthetic resampling noise, not leakage; it caps the achievable RMSE (an irreducible noise floor) and punishes high-capacity models that try to memorize individual rows. The strongest EDA signals, all consistent with concrete chemistry, were:

- **AgeInDays** is the dominant driver (Pearson 0.334, Spearman 0.604) but heavily right-skewed (skew 2.75); `log1p(AgeInDays)` correlates far more strongly and near-linearly with `Strength` (Pearson 0.558) — strength develops roughly log-linearly with curing time.
- **Supplementary cementitious materials are zero-inflated:** `BlastFurnaceSlag` is 0 in 58.6% of rows, `FlyAshComponent` in 72.6% — present only in some mixes, not missing.
- **Water/total-binder ratio** (binder = Cement + Slag + FlyAsh) is a stronger signal (Pearson -0.227) than the naive water/cement ratio (Pearson -0.151), because slag and fly ash also consume water in hydration.

- **SuperplasticizerComponent** (Pearson 0.208) and **CementComponent** (Pearson 0.158) are the next strongest raw drivers after age.

Annotation Guidelines. The label is `Strength` $\in \mathbb{R}$ (mean 35.45, std 16.40, range 2.33–82.60, 843 unique values). Submissions are **real-valued strength predictions** scored by RMSE — a point-estimate regression, not a ranking or a class decision.

Feature Set. From the 8 raw components we engineer **22 features**, grouped below:

Group	Features
Raw components (8)	<code>CementComponent</code> , <code>BlastFurnaceSlag</code> , <code>FlyAshComponent</code> , <code>WaterComponent</code> , <code>SuperplasticizerComponent</code> , <code>CoarseAggregateComponent</code> , <code>FineAggregateComponent</code> , <code>AgeInDays</code>
Age transforms	<code>log_age</code> , <code>sqrt_age</code>
Zero-inflation flags	<code>has_slag</code> , <code>has_flyash</code> , <code>has_superplasticizer</code>
Binder / mass	<code>binder</code> (Cement+Slag+FlyAsh), <code>total_mass</code>
Ratios	<code>water_binder_ratio</code> , <code>water_cement_ratio</code> , <code>sp_binder_ratio</code> , <code>agg_binder_ratio</code> , <code>fine_coarse_ratio</code> , <code>slag_cement_ratio</code> , <code>flyash_cement_ratio</code>

Splitting strategy. A single canonical **5-fold StratifiedKFold on Strength deciles** (`pd.qcut(y, 10)`, shuffle, `seed = 42`), shared across every experiment so scores are directly comparable. Decile-stratified folds were chosen over plain KFold for more stable splits on a small (5.4k-row), noisy regression target. The leaderboard was **not submitted this run** (no Kaggle credentials in-session), so all scores here are **CV-only**; no CV $\leftrightarrow$ LB comparison is available (`leaderboard` is recorded as null / missing in `facts.json`).

Models & Architecture

Purpose of Architecture. A regressor that minimizes RMSE against a label-noise-bounded target. **Architecture Type** is a **gradient-boosted decision-tree ensemble** — CatBoost as the primary (strongest solo) learner, with LightGBM and XGBoost members and their seed-bagged variants, combined by a weighted blend; the committed champion is a **7-member seed-bagged blend** (experiment #8), which the tree search rediscovers as its **node #13, a 13-way blend of the first-generation solo pool**.

Each member takes the same **Input Format** — a purely numeric feature matrix — of **Input Dimension 22 features** per row (a 1-D vector; there is no spatial/sequence structure).

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by gradient boosting (CatBoost’s ordered/symmetric trees; LightGBM’s histogram-based leaf-wise trees). The champion CatBoost member is `depth = 6`, `l2_leaf_reg = 6`, up to 4,000 iterations at `learning_rate = 0.03`. The blend layer sits on top as a convex weighted average of the members’ out-of-fold predictions, with weights found by Dirichlet sampling plus coordinate-descent refinement. **Model Complexity** is best expressed as `trees  $\times$  depth` rather than a dense parameter count: a depth-6 CatBoost member is $2^6 = 64$ -leaf symmetric trees over up to 4,000 iterations, and the champion blends up to 13 such members (7 in the committed exp #8 pool). CatBoost’s ordered-boosting regularization is the reason it resists the 56%-duplicate-row noise ceiling

better than LightGBM/XGBoost at default capacity — under generic untuned settings the weight search assigned CatBoost 100% and zeroed the other two.

Training procedures

Training proceeds as a **tiered progression** on the shared 5-fold split, each tier measured on the same OOF scale:

Tier	Configuration	OOF RMSE
1	generic untuned 3-model blend (raw 8 features)	12.54287
2	skill pipeline: 22 features + regularized LGB/XGB/CAT, 3-way weighted blend	12.073474
3	+ linear self-improvement (Optuna-tuned LGB + seed-bagging), 7-way blend — committed champion	12.07003
4	+ tree-search <code>harness_v3</code> (26 nodes; node #13 = 13-way blend of the solo pool)	ties 12.07003

The **Loss Function** is squared error (RMSE objective); model selection is on OOF RMSE. A notable negative ablation: swapping to a **robust loss (huber/fair)** made **LightGBM substantially worse** (12.11 → 12.16–12.22), the opposite of the a-priori hypothesis that a residual-magnitude-insensitive loss would help under the label-noise ceiling — GBDT with squared loss already fits the conditional mean of duplicated rows. The **Optimization Algorithm** is gradient boosting (CatBoost ordered/symmetric; LightGBM histogram leaf-wise) — **not** SGD/ADAM — with blend weights optimized by Dirichlet sampling + coordinate descent. The **Learning Rate** is the boosting shrinkage (champion CatBoost `learning_rate` = 0.03 ; the Optuna-tuned LGB member used 0.0198); there is **no Learning Rate Scheduler** (N/A — *GBDT convergence is governed by CV early-stopping, not a schedule*) and **no Batch Size** (N/A — *full-dataset histogram/ordered boosting, not mini-batched*).

Training Duration for the tree-search stage was **~125 s of wall-clock over 20 model fits** (the harness run stats; the v3 tree is a faithful replay that reuses cached OOF, so its per-node cost is ~0 s). A solo CatBoost fits in ~2.7 s and the Optuna tune of the LGB member ran 60 TPE trials in 371.6 s. **Training Memory** was not explicitly capped — a $5,407 \times 22$ float matrix is trivial on the single arm64 CPU machine; peak was **not recorded**. **Transfer Learning** is N/A (*no pretrained weights*); its analogue here is **cross-season experience injection** — the tree search cites distilled priors (P5: CatBoost’s ordered boosting resists label noise; P8/P13: seed-bagging is the only reliable post-plateau lever) carried in from earlier episodes. **Data Augmentation** is N/A (*tabular*); the analogues are feature engineering (the 22 features) and seed-bagging. An explicitly *rejected* augmentation was fold-safe duplicate-group target smoothing (exp #5), which made the blend worse (+0.00775) because the squared-loss minimizer already prices in the group mean.

Reproducibility Standards are strict: a fixed fold seed (42), logged Optuna and seed-bag seeds, and legacy-OOF reuse verified digit-for-digit against `STATUS.md`’s historical values before searching. Honesty note on the v3 faithful replay: **21 of 26 nodes reproduced exactly, but the champion node drifted by $\Delta = -0.003$** (12.07003 → 12.066947) under **early-batch LightGBM nondeterminism** — a replay artifact, not a real gain, so the reported champion remains the committed **12.07003**.

Inference procedures

Decision Threshold is *N/A* — RMSE is a point-estimate regression metric, so we emit raw real-valued strength predictions and never threshold. **Inference Duration** was **not separately profiled**; GBDT scoring of the 3,605 test rows is sub-second per member on CPU. **Inference Memory** is negligible (tree inference is lightweight, **not recorded** as capped).

Evaluation & Benchmarking

Task for Performance Evaluation. Predict the compressive strength of each test mix. **Performance Metrics.** RMSE is the sole competition metric (minimize); our committed champion reaches **CV OOF RMSE 12.07003**, improving $12.54287 \rightarrow 12.073474 \rightarrow 12.071429 \rightarrow 12.07003$ across the baseline, skill-pipeline, and two seed-bagging rounds — a 0.47284 (3.77% relative) gain over the generic baseline. The gains after Tier 2 came almost entirely from seed-bagging (variance reduction at the noise ceiling), not from tuning or denoising, both of which were tried and failed. The run was **not submitted to Kaggle**, so no LB anchor exists.

Performance Benchmarking. The natural comparator is the **NVIDIA reproduce-agent**, which copies the strongest public kernel verbatim:

Agent	Approach	CV RMSE	Note
NVIDIA	reproduces <code>ambrosm/pss3e9-winning-model</code> (verbatim)	12.026	marginally lower
Our agent	from-scratch GBDT pool + tree-search	12.07003	+0.044 (0.37% higher)

NVIDIA’s 12.026 is marginally below our 12.07003 (RMSE, lower is better), but the four-axis evaluation scores the pair a **tie** — the ~0.044 RMSE gap is within the noise floor this competition’s 56%-duplicate-row label structure imposes, so a verbatim copy of the public winning kernel and our from-scratch originate-agent land in the same place. On this label-noise-capped, small-data episode there is no meaningful daylight between reproducing the best public solution and building one from scratch.

Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08

All 20 baseline competitions are being re-run in an isolated root (`~/benchruns/myagent-rerun`) under one frozen architecture (`docs/rerun_manifest.json`): every lane runs the **full v5 pipeline including the Stage 0.5 problem dossier** (dossier.json written in-lane before modeling), holds no credentials, and is checked by a per-lane transcript audit. Leaderboard scores for the re-run are **TBD** until the separate credentialed operator scoring step; the figures below are the lane’s own local CV only and revise nothing in the sections above.

- **Lane:** done 2026-08-14, 28 min.
- re-run local CV: **RMSE 12.082282** honest leave-fold-out (12.075202 fitted-OOF), 3-member CatBoost-family NNLS blend.
- Public / private leaderboard for this re-run: **TBD** (pending operator scoring).